



南京大學

本科畢業論文

學院 智能軟件與工程學院

專業 軟件工程（智能化軟件）

題目 面向實踐的緊湊哈希表設計與
簡化復現

年級 2022 學 號 221900323

學生姓名 紀澤操

指導教師 劉明謀 職 稱 副教授

提交日期 2026 年 5 月 30 日



南京大学本科毕业论文（设计） 诚信承诺书

本人郑重承诺：所呈交的毕业论文（设计）（题目：面向实践的紧凑哈希表设计与简化复现）是在指导教师的指导下严格按照学校和学院有关规定由本人独立完成的。本毕业论文（设计）中引用他人观点及参考资源的内容均已标注引用，如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文（设计）等违纪行为。

作者签名：纪泽操

学号：221900323

日期：2026 年 5 月 30 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：面向实践的紧凑哈希表设计与简化复现

学院：智能软件与工程学院

专业：软件工程（智能化软件）

本科生姓名：纪泽操

指导教师（姓名、职称）：刘明谋 副教授

摘要：

本论文以《On the Optimal Time/Space Trade off for Hash Tables》为实验方向，系统整理文中提及的理论算法及核心思想，然后加以简化并落实到实际当中，从宏观架构层面，搭建了 Facility - Cubby - Slot 三层存储体系，从而达成高效地组织和运作哈希表的目的；微观机制层面，融合了可逆哈希函数，局部查询路径，商压缩，K - Kick 规则，多层级 Cubby 以及双表渐进式扩缩容迁移等主要技术。落地的哈希系统包含初始化，插入，删除，查询，扩容以及性能指标验证等完整的操作，实验结果表明，该系统可以稳定执行各类动态操作，其路由访问步数，元素被踢出的移动次数以及元数据编码位数等关键指标均处于可控制范围内，能够精准再现原始理论架构的核心逻辑，而且，本文明确了工程简化达成与严格理论模型之间的差异界限，给类似紧凑哈希结构的工程化重现及其在实际场景中的应用赋予了有价值的参考。

关键词：哈希表；时间空间权衡；探测复杂度；增强开放寻址；k-kick 树；动态扩容

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Practical Compact Hash Table Design and Simplified Reproduction

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software Engineering (Intelligent Software)

UNDERGRADUATE: Ji Zecao

MENTOR: Liu Mingmou, Associate Professor

ABSTRACT:

This thesis is built on the study of the paper *On the Optimal Time/Space Tradeoff for Hash Tables*. It does an analysis on the theoretical algorithms and main ideas in the paper, and then tries to make them work in real engineering by using a more simple and practical way. From the overall structure point of view, a three-layer storage model made up of Facility, Cubby, and Slot is designed, which helps to manage the hash table in a more organized way. At the implementation side, the system uses many key techniques like reversible hash functions, localized query paths, quotient compression, the K-Kick rule, multi-tier Cubby organization, and incremental dual-table resizing and migration. The built hash system can support all basic functions such as initialization, insertion, deletion, lookup, dynamic resizing, and performance testing. The experiment results show that it can handle different dynamic operations stably, and important indicators like routing probe counts, kick displacement frequency, and metadata encoding overhead are kept in acceptable levels. The implementation is able to get the core behavior of the original theoretical design in a reasonable way. Also, this work makes a discussion about the differences between the engineering simplifications used here and the pure theoretical model, which can be helpful for the practical rebuilding and use of compact hash table structures.

KEYWORDS: hash tables; time–space tradeoff; the Probe-Complexity problem; enhanced open addressing; k-kick tree; dynamic expansion

目 录

目 录	III
第一章 引言	1
1.1 实验背景及意义	1
1.2 系统结构概要	2
1.3 国内外现状	2
1.4 实验路线	3
第二章 问题模型与理论基础	4
2.1 探测复杂度与球槽分配模型	4
2.2 系统核心设定：全域与哈希基础	5
2.2.1 全局参数	5
2.2.2 可逆置换与全局路由哈希	5
2.2.3 Facility 编号与 Cubby 内局部地址	6
2.3 核心组件层级关系	6
2.4 商压缩的信息论思想	7
2.5 k-kick、MiniArray 与 Local Query Router 的理论角色	8
2.5.1 k-kick：逻辑分层与交换上界	8
2.5.2 MiniArray M ：空闲 bin 的常数时间判定	9
2.5.3 Local Query Router： $O(1)$ 查询与映射存储	9
2.5.4 辅助结构同步不变量	9
2.6 本章小结	10
第三章 哈希表结构与算法设计	11
3.1 Facility 与 MiniArray	11

3.1.1	Facility 的基本属性	11
3.1.2	MiniArray 的逻辑结构	11
3.1.3	Access: 按逻辑下标读取	12
3.1.4	Insert: 插入与叶分裂	12
3.1.5	Delete: 删除与叶合并	12
3.1.6	Update: 原地覆写	14
3.1.7	Split 与 Merge	14
3.1.8	Rank 与 Select	14
3.1.9	复杂度与空间性质	15
3.2	多层次 Cubby 设计	15
3.2.1	层级属性与初始状态	15
3.2.2	动态调整: 合并与拆分	16
3.2.3	Cubby 内部组件	16
3.2.4	插入与删除操作	17
3.3	k-kick 层级插入算法	17
3.3.1	层级 bin 定义	18
3.3.2	偏好序列	18
3.3.3	探测位置序列	19
3.3.4	bin 饱和与最大可用深度	19
3.3.5	随机深度选择与 InsertKick	20
3.3.6	ReInsert: 被踢元素向上安置	20
3.3.7	复杂度与正确性	22
3.4	Local Query Router	22
3.4.1	设计目标	22
3.4.2	映射规则	22
3.4.3	二叉前缀树存储与接口	23
3.4.4	复杂度与 Patricia 压缩	23
3.5	插入、查询与删除的统一语义	25
3.6	本章小结	25

第四章 系统性能实验	26
4.1 实验目的与环境	26
4.2 实验方案	26
4.2.1 测试程序与工作量	26
4.2.2 评价指标	27
4.3 宏基准实验	27
4.3.1 实验配置	27
4.3.2 各阶段延迟	28
4.3.3 终态结构指标	28
4.4 多预设参数扫描	29
4.5 补充实验	29
4.5.1 较大规模插入	29
4.5.2 随机种子对比	30
4.5.3 逻辑元数据与槽位占用对照	30
4.6 实验结果分析	31
4.6.1 数据规模与查询延迟	31
4.6.2 Facility 规模参数 K 的影响	31
4.6.3 k-kick 深度 k 的影响	31
4.6.4 负载因子与扩缩容	32
4.6.5 tier 重建与双表迁移	32
4.7 本章小结	32
第五章 总结与展望	33
5.1 工作总结	33
5.2 不足与局限	34
5.3 展望	34
参考文献	35
致 谢	39

第一章 引言

1.1 实验背景及意义

哈希表是一种通过哈希函数把关键字映射到数组相应位置的结构^[1-2]，它在数据库索引、键值存储、编译器符号表以及运行时系统等常见基础架构中被广泛应用。相较于链地址法，开放寻址哈希表将元素直接放置在连续数组中，其查询路径相对较短，缓存局部性表现更优；但是当负载因子上升时，冲突及探测长度会快速影响操作成本。在实际工程操作中，常采用预留空槽的方式以降低冲突发生几率，但这也会致使单位元素所占据的附加空间有所增加。因此哈希表的设计需解决如下具体问题：在确保查询、插入及删除操作时间可有效控制的基础上，如何减少每个元素为实现定位、更新与恢复而额外存储的信息量。

Bender 等人等人的论文^[3]从理论层面阐述了上述问题：在只能花费常数时间执行查询的前提下，若插入和删除允许承担 $O(k)$ 次元素交换代价¹，则每个元素所占的额外空间可从 $O(\log \log n)$ 比特降至 $O(\log^{(k)} n)$ 比特²。该结果表明，开放寻址哈希表的空间利用率不仅取决于空位占比，还与元素搬移、局部引导及键重建信息的编码方式密切相关。此前，紧凑哈希表在常数时间操作下每键约需 $O(\log \log n)$ 比特冗余；动态检索与 filter 的下界^[4]、Iceberg 系列结果^[5-6] 以及紧凑检索结构^[7] 等已逐步逼近信息论下界。本文在该理论框架下给出完整实现与性能评估：Facility 级 MiniArray、k-kick 层级插入与 Local Query Router 均按原论文语义落地；具体架构与模块分工见下一节，各模块的算法细节留待第二、三章展开。

1 一次更新过程中被搬移（踢出或回填）的元素个数。

2 本文空间开销均按比特计； $\log^{(k)} n$ 表示对 n 连续取 k 次对数。

1.2 系统结构概要

为避免后文术语堆砌以及便于理解，本节仅从宏观上勾勒本文实现所采用的分层组织；各模块的算法语义与代码路径不在此赘述。

从逻辑结构上看，我们可以将整张表视为一个规模为 N 的开放寻址结构。实现上按地址将其划分为若干 Facility（表级分区），每个 Facility 内包含若干 Cubby（局部存储块）及配套的 MiniArray 元数据（用于存储数据长度不同的结构）。元素最终落在 Cubby 的槽位上：槽位只保存经商压缩剥离寻址低位后的载荷比特；恢复完整键所需的变长信息由 Facility 级 MiniArray 与 Cubby 内 *MiniArray(B)* 另行编码。插入操作会在 Cubby 内沿 k-kick 规则做有界迁移，并由 MiniArray M 维护 bin（k-kick 树种将不同层级的区域称为 bin）空闲状态；查询时 Local Query Router（ $A[i]$ 前缀树）直接给出探测下标，无需扫描完整的探测序列。哈希表规模变化时通过 active/old 双表在后续操作中按预算逐步迁移，避免一次性全表重哈希。

可将上述名称理解为“分区—局部块—槽位”的三级骨架，以及配套的局部定位、压缩存储与渐进扩缩容手段；具体实现在后文展开，后文直接使用这些术语，不再逐条给出形式化定义。

1.3 国内外现状

哈希表实现主要包括链地址法与开放寻址法两大类^[1-2]。链地址法借助链表或桶结构处理冲突，实现直接，但指针与动态分配会带来额外空间和缓存开销。开放寻址法将元素置于数组槽位，局部性较好，却易受负载因子、探针序列与删除策略影响。布谷鸟哈希^[8-9]以多候选位置与踢出过程缩短查询路径；最小完美哈希^[10-12]面向静态集合提供更强查找保证，其动态扩展与紧凑编码亦有持续研究^[4,13]；打包与压缩哈希表^[14]、IcebergHT^[5-6] 等工程方案则侧重实际吞吐与空间折中。这些路线分别改进了查询确定性、构造效率或空间利用率，但在“动态插入删除”与“极低附加比特”之间仍难以同时兼顾。

近年来，研究进一步聚焦到位级空间开销：仅靠负载因子无法刻画紧凑哈希表的真实成本，因为系统还须保存支持查询、更新与键恢复的辅助结构。Bender 等人的最优时间/空间权衡框架^[3]将元素放置建模为带删除的球槽分配^[15-16]，并

以 k-kick 树、局部查询路由器 (Local Query Router)、MiniArray 与商压缩组合出可调的权衡曲线。本文工作定位于该框架的完整工程实现与实验观察：在 Facility-Cubby-Slot 分层组织下，将 MiniArray 变长元数据维护、k-kick 有界交换插入与 Local Query Router 常数时间查询三条主线贯通，并通过延迟、踢出强度与 router 深度等指标检验理论权衡曲线在实测中的可观测性^[3-4]。

1.4 实验路线

本文以 Bender 等人的论文^[3]的关键构造为核心，给出完整实现并进行实验评估。实现沿用 Facility-Cubby-Slot 组织：Facility 级 MiniArray 维护变长元数据；Cubby 内 k-kick 完成有界交换插入；Local Query Router 将查询降为 $O(1)$ ；槽位存放商压缩载荷，Cubby 内部的 MiniArray B 保存键恢复差分；扩缩容采用 active/old 双表渐进迁移。实验重点覆盖插入、查询、删除与迁移路径，并统计踢出次数、元数据位数等指标。

本文针对三个相互联系的问题依次展开论述：

1. 在 MiniArray、k-kick 与 Local Query Router 完整实现的前提下，插入、查询与删除各阶段的延迟与吞吐处于何种量级。
2. 插入迁移次数、kick/ins 与扩缩容事件如何随 n 、 K 、 k 与负载因子变化，是否与 $O(k)$ 交换上界及 $O(\log^{(k)} n)$ 空间目标相一致。
3. 当前实验 workload 与测量口径在哪些方面尚不足以覆盖 tier 重建、超大规模长跑及外部对照，从而限制结论的外推范围。

后续章节安排如下：第二章介绍理论基础，第三章阐述哈希表结构与算法设计，第四章报告系统性能实验结果，第五章总结工作并讨论改进方向。

第二章 问题模型与理论基础

2.1 探测复杂度与球槽分配模型

开放寻址哈希表可被抽象为在线状态下的球槽分配问题^[15-17]：表中有固定数量的槽位，每个槽位至多容纳一个元素；系统支持动态插入与删除，且任一时刻元素个数不超过槽位数。对任意键 x ，哈希过程为其生成一条预先确定的随机探测序列

$$h_1(x), h_2(x), \dots$$

元素只会被安置于该序列给出的候选槽位之一。

当某元素最终被置于其探测序列的第 i 个候选位置时，探测复杂度¹定义为 $\Theta(1 + \log i)$ 。该量可理解为查询阶段为记录“元素采用了第几个候选位置”而必须保存的比特数目，因而与辅助元数据的空间开销直接相关。在该模型下，平均探测复杂度刻画整体附加空间，交换成本刻画单次更新中被搬移元素的个数；最优哈希表理论所关注的核心问题，是在允许有限交换的前提下，如何尽可能降低平均探测复杂度^[3-4,18]。形式地，若表内 n 个元素分别落在其探测序列的第 $i_1, i_2, \dots, i_n$ 个候选位置，则平均探测复杂度为

$$\frac{1}{n} \sum_{t=1}^n \Theta(1 + \log i_t),$$

而单次插入或删除的交换成本为踢出链中被搬移元素个数的上界。Bender 等人^[3]通过调节 k-kick 深度参数 k ，使上述两项分别控制在 $O(\log^{(k)} n)$ 与 $O(k)$ 量级，形成可调的时间/空间权衡曲线。更精确地，他们提出：若查询必须在常数时间内完成，且插入、删除允许承担 $O(k)$ 次元素交换，则每个元素所需的额外空间可由 $O(\log \log n)$ 比特降至 $O(\log^{(k)} n)$ 比特。该结果说明，开放寻址哈希表的空间效

¹ 查询时需记录“元素落在第 i 个候选位置”所需的比特数，记为 $\Theta(1 + \log i)$ ，与辅助元数据空间直接相关。

率不仅取决于空位占比，还与元素搬移、局部引导信息及键恢复编码密切相关。后文将给出与之配套的地址拆分、分层存储与 Local Query Router 设计，使探测序列在插入阶段完整定义、在查询阶段却不必线性扫描。

2.2 系统核心设定：全域与哈希基础

2.2.1 全局参数

设键空间全域为 U ，且 $|U| = \text{poly}(n)$ 。当前表内元素规模提示为 n ；表容量 N 取为 2 的幂，使实际容量落在区间 $[N, 2N]$ 内。每个 Facility 的基准规模为

$$K = \text{polylog } n,$$

通常取 $K = \Theta(\log^c n)$ 且 $c \in [2, 4]$ ，以便在 Facility 数量 N/K 与局部结构规模之间取得平衡^[2-3]。

2.2.2 可逆置换与全局路由哈希

对任意键 x ，先经随机可逆置换 π 映射为 $\pi(x)$ 。该类置换可由 Feistel 等构造从伪随机函数生成^[19-21]。定义 $x[a, b]$ 表示 x 的第 a 位到第 b 位组成的子串（从低位到高位），则全局路由哈希 $g^*(x)$ 定义为 $\pi(x)$ 的低 $\log N$ 位：

$$g^*(x) = \pi(x)[1, \log N].$$

置换值在 $\log N$ 位以上的部分存入槽位载荷，记为

$$\text{storage} = \pi(x)[\log N + 1, \log U],$$

即商压缩意义下的 payload；该部分不再参与 Facility 与 Cubby 的路由，仅在命中校验时与局部元数据联立以恢复完整 $\pi(x)$ 。

从比特布局上看， $\pi(x)$ 可示意划分为“高位 payload + 低位 $g^*(x)$ ”；而 $g^*(x)$ 自身又可自高向低继续划分为 Facility 编号段、Cubby 中间段与 local query router 段，分别用于表级分区、Cubby 内桶化与局部 router 索引。图 2-1 概括了这一层

次。

$\pi(x)$ 高位: payload / storage	$g^*(x) = \pi(x)[1, \log N]$	
$\pi(x)[\log N + 1, \log U]$	Facility 位	Cubby 中间位 + $g_I(x)$
存入槽位 storage	$r = g^*[\log K + 1, \log N]$	$g^*[1, \log K]$

图 2-1 $\pi(x)$ 与 $g^*(x)$ 的位段划分（示意）

2.2.3 Facility 编号与 Cubby 内局部地址

在 $g^*(x)$ 已确定的前提下，Facility 编号取

$$r = g^*(x)[\log K + 1, \log N] = \left\lfloor \frac{g^*(x)}{K} \right\rfloor.$$

对容量为 $|I|$ 的 Cubby，其内部 local query router 段（与 k-kick 偏好序列无关）定义为

$$g_I(x) = g^*(x)[1, \log |I|],$$

即 $g^*(x)$ 的最低 $\log |I|$ 位，取值范围为 $[0, |I| - 1]$ 。所有满足 $g_I(x) = i$ 的键由该 Cubby 内第 i 号 local query router $A[i]$ 管理； $g^*(x)$ 中 $[\log |I| + 1, \log K]$ 的中间位则与 Facility 编号 r 及 payload 共同用于恢复完整 $g^*(x)$ ，进而经由 π^{-1} 得到原始键 $x^{[3,7]}$ 。

2.3 核心组件层级关系

在逻辑结构上，整张表可视为一个大规模数组，均等划分为 (N/K) 个 Facility；每个 Facility 管理规模为 K 的局部范围，并在其内部维护多层级、多容量的 Cubby 集合。Cubby 是实际存放元素的局部块；每个 Cubby 内部又包含以下部分：

- storage 数组：长度为 $|I|$ 的槽位数组，存放商压缩 payload 即 $\pi(x)[\log N + 1, \log U]$ ；
- k-kick 逻辑：将整个 cubby 逻辑上划分为多 bin 层级，完成插入、踢出与被踢元素的重插，不要求额外的物理树结构；
- MiniArray A ：长度为 $|I|$ 的 local query router 数组， $A[i]$ 维护所有 $g_I(x) = i$ 的键到探测序列下标的映射；

- MiniArray M : 记录各 k-kick 深度 bin 的空闲槽位信息, 支持在常数时间内判定 bin 内是否存在空位、定位空位;
- MiniArray B : 与 storage 槽位一一对应, 保存 $\delta = g_I(x) - i$ 及恢复 $g^*(x)[\log |I| + 1, \log K]$ 所需的中间位等信息。

Facility 层另维护一棵基于 B 树、深度为 $O(1)$ 的 MiniArray, 用于在变长元数据与 Facility 级索引上实现均摊常数时间的访问、插入、删除与更新^[3,22]。MiniArray 中每个内部节点最多有多少个子节点称为 fanout, 取 $\Theta(\log n / \log \log n)$, 内部指针仅需 $\Theta(\log \log n)$ 比特, 从而与整体 $O(\log^{(k)} n)$ 的空间目标相容^[23-24]。

为在动态扩缩容下维持高装填因子, 每个 Facility 还引入 *tail Cubby* 机制: 初始时仅含一个 1-tiered Cubby 作为 tail; 此后所有新插入默认进入 tail, tail 满后转为普通 1-tiered Cubby 并新建 tail。不同 j -tiered Cubby 的目标个数与容量分别为

$$t_j = \Theta\left(\frac{(\log^{(j+1)} n)^2}{(\log^{(j)} n)^2}\right), \quad r_j = \frac{K}{(\log^{(j)} n)^2};$$

当某层 Cubby 数量偏离 t_j 时, 通过向上合并或向下拆分并在新 Cubby 内重插全部元素来恢复分布不变量。删除非 tail 元素时, 从 tail 随机抽取一个元素回填至被删槽位; tail 为空时从 1-tiered 层提升一个 Cubby 作为新 tail, 并可能触发拆分。tier 调度、tail 不变量与 k-kick 踢出规则共同构成第三章的算法主体^[2-3,6,25]。

2.4 商压缩的信息论思想

若于槽位中完整保存原始键, 则部分信息与寻址过程重复。商压缩借助双射函数 π , 映射 x 为 $\pi(x)$, 低 $\log N$ 位 $\pi(x)[1, \log N] = g^*(x)$ 的各位段交由地址上下文 (Facility 编号、Cubby 容量、槽位下标) 与局部元数据分担, 槽位仅保存 payload 即 $\pi(x)[\log N + 1, \log U]$ 及无法由上下文恢复的差分信息。

具体地, 当 $\text{storage}[i]$ 存放键 x 对应信息时, MiniArray $B[i]$ 保存

$$\delta = g_I(x) - i,$$

以及 $g^*(x)[\log |I| + 1, \log K]$ 的中间位。配合 Facility 编号 $r = g^*(x)[\log K + 1, \log N]$

，即可重建

$$g^*(x)[1, \log N],$$

再加上 payload 得到 $\pi(x)$ ，经 π^{-1} 恢复 x 。该编码方式与紧凑检索结构^[7,10]、打包式哈希表^[12,14] 所强调的“按条目实际长度计费”一脉相承；但在 k-kick 动态搬移与 tier 重建下， B 、 A 与 M 必须同步更新，对 MiniArray 与 Local Query Router 的在线维护提出了较静态紧凑哈希更严格的要求^[3-4]。

2.5 k-kick、MiniArray 与 Local Query Router 的理论角色

2.5.1 k-kick：逻辑分层与交换上界

在 Cubby 内部，k-kick 并不引入物理上的多层数组，而是将 storage 上长度为 $|I|$ 的连续槽位按固定规则划分为 $k + 1$ 个逻辑深度：深度 0 的 bin（bin 指的是 Cubby 按不同深度划分的子区间）覆盖整个 Cubby，更深层 bin 由上一层均匀切分得到，规模 $s_i = \Theta((\log^{(i)} K)^6)$ 。对键 x （此处均指 $\pi(x)$ ）定义偏好序列

$$g_0(x) \rightarrow g_1(x) \rightarrow \cdots \rightarrow g_k(x),$$

其中 $g_0(x)$ 为整个 Cubby， $g_{i+1}(x)$ 为 $g_i(x)$ 经均匀切分后 x 所属的子 bin。探测位置序列 $h_1(x), h_2(x), \dots$ 按先深后浅顺序枚举各 bin 内槽位；若元素最终落在第 j 个候选位置，则其探测复杂度为 $\Theta(1 + \log j)$ 。

定义 Cubby 某一个 bin 饱和 IsSaturated 的定义为：bin 已满且其中每个元素的 $\text{insert_depth} \geq \text{bin}$ 对应的深度，贪心思维下函数 $\text{GetMaxUsableDepth}(x)$ 为 x 可以插入的且未饱和的最大的 bin 的深度。插入时，系统先取随机哈希 $s(x) \in [0, k]$ ，再令

$$\text{depth} = \min(\text{GETMAXUSABLEDEPTH}(x), s(x)),$$

在 bin $g_{\text{depth}}(x)$ 内写入；若 bin 已满，则踢出其中 insert_depth 最小的元素并调用 ReInsert 向上安置。深度 d 下 bin 饱和的定义为：bin 已满且其中每个元素的 $\text{insert_depth} \geq d$ 。Bender 等人^[3]证明：在参数满足其设定时，整条踢出链至多 k 次，单次插入的交换成本为 $O(k)$ ；配合随机深度选择，平均探测复杂度可控制

在 $O(\log^{(k)} n)$ 量级，从而与上述描述的空间—时间权衡曲线对齐^[8-9,18]。

2.5.2 MiniArray M ：空闲 bin 的常数时间判定

k-kick 的 IsSaturated、GetMaxUsableDepth 与 ReInsert 均需在 bin 粒度上判定“是否含空位”并定位空位。MiniArray M 为各深度 bin 维护等价于 bitmap 的空闲槽位图，使上述判定与定位在均摊意义下为常数时间，而不必扫描整个 Cubby^[3,18]。Facility 级 MiniArray 则通过 $O(1)$ 深度 B 树、叶内 Rank/Select 与 Packed Leaf，在变长元数据上实现均摊常数时间的 Access/Insert/Delete/Update^[22-24]：树高 $O(1)$ ，内部指针 $\Theta(\log \log n)$ 比特，叶分裂/合并保证均摊平摊^[11,14]。

2.5.3 Local Query Router： $O(1)$ 查询与映射存储

有了上述的上述的偏好序列和探测位置序列，我们还需要考虑如何让查询做到 $O(1)$ 的时间复杂度，不遍历整个探测序列 $h_1(x), h_2(x), \dots$ ，因为 k-kick 插入会 kick 元素，元素实际位置 $\neq$ 首选位置 $g_k(x)$ ，也不等于 $h_1(x)$ ，插入用 k-kick 动态挪动了位置，查询如果再按 $h_1(x), h_2(x), \dots$ 扫过去，就不是 $O(1)$ 的时间复杂度。Local Query Router 因此为每个键保存精准映射

$$\pi(x) \longrightarrow j,$$

其中 j 为探测序列下标。Cubby 内令 $g_I(x) = g^*(x)[1, \log |I|]$ ，所有满足 $g_I(x) = i$ 的键由 router $A[i]$ 管理； $A[i]$ 实现为二叉前缀树（可进一步 Patricia 压缩^[13,26]），对外接口为 insert(key, j)、query(key)、delete(key)。查询时先算 $i = g_I(x)$ ，在 $A[i]$ 得 j ，再访问 $h_j(x)$ 并用 B 与 payload 校验； $j \leq \log K$ ，每条映射仅需 $O(\log \log n)$ 比特，查询路径为常数次 router 与 MiniArray 访问^[3,7]。

2.5.4 辅助结构同步不变量

元素在 k-kick 踢出链中每发生一次搬移，须同步更新四处状态：

- storage 槽位内容与 insert_depth；
- $B[i]$ 中的 δ 与中间位；

- $A[g_I(x)]$ 中 $\pi(x) \mapsto j$ 的映射；
- M 中各 bin 的空闲位图。

删除非 tail 元素时的 tail 随机回填、tier 合并/拆分后的全量重插，以及表级双表扩缩容，均遵循同一同步规则——不可仅复制 payload 而忽略 router 与 B 的上下文依赖^[2-3]。第三章将给出 MiniArray、k-kick 与 Local Query Router 的全部算法伪代码及统一操作语义。

2.6 本章小结

本章建立了后文所需的理论语境：球槽分配模型给出探测复杂度与交换成本的定义^[1,15]；全域参数与 $g^*(x)$ 位段划分明确了地址语义；Facility–Cubby–storage– $A/M/B$ 层级关系给出整体骨架；商压缩说明 payload 与 $B[i]$ 的信息分工^[3,13]；同时从 k-kick 交换上界、MiniArray M 的空闲判定、Local Query Router 的 $O(1)$ 查询及四路同步不变量四个角度，为第三章算法设计提供理论锚点。下一章将给出各模块的完整伪代码；第四章则报告宏基准与参数扫描下的性能测量。

第三章 哈希表结构与算法设计

本章依照系统核心设定，给出分层哈希表各组件的完整结构与算法细节。内容覆盖 Facility 级 MiniArray、多层级 Cubby 组织、Cubby 内 k-kick 插入与 Local Query Router，并在最后一节统一说明插入、查询与删除的操作语义。

3.1 Facility 与 MiniArray

3.1.1 Facility 的基本属性

每个 Facility 对应全局表中的一个局部片段，规模为 $K = \text{polylog } N$ 。Facility 管理其内部的全部 Cubby（含各 tier 及唯一 tail），并维护 Facility 级 MiniArray，用于在变长元数据上提供均摊常数时间的 Access、Insert、Delete 与 Update 等函数。B 树的 fanout(上文有定义，fanout 为每个内部节点的子指针个数) 记为 F ，取 $F = \Theta(\log n / \log \log n)$ ；树高为 $O(1)$ ，每个内部节点的子指针仅需 $\Theta(\log \log n)$ 比特^[3,22-23]。

3.1.2 MiniArray 的逻辑结构

MiniArray 由以下部分共同组成：

MiniArray $\rightarrow O(1)$ 深度 B 树 $\rightarrow$ Packed Leaf + Bitmap + Rank/Select + Lookup Table

叶节点 Leaf 包含：

- bitmap：标记叶内哪些逻辑位置已有有效元素；
- data：PackedArray，在有效位置上紧凑、无间隙地存放变长元素。

内部节点 Internal 包含：

- subtree_size[F]：每个子树中有效元素总数，供 Rank/Select 定位；
- children[F]：至多 F 个子女指针，每个指针 $\Theta(\log \log n)$ 比特。

对外接口为 $\text{Access}(\text{idx})$ 、 $\text{Insert}(\text{idx}, \text{val})$ 、 $\text{Delete}(\text{idx})$ 、 $\text{Update}(\text{idx}, \text{val})$ ，其中 idx 为逻辑下标而非物理槽位号。叶容量上限记为 MAX_LEAF_SIZE ，由 NODE_MAX_BITS 常数 $c \in [1, 3]$ 派生^[3,11,14]。

3.1.3 Access：按逻辑下标读取

Access 自根向下，利用各层 subtree_size 将逻辑下标映射到目标叶节点；在叶内用 Select 于 bitmap 上定位第 idx 个有效位置，再从 data 读出元素。树高为 $O(1)$ ，叶内 Select 通过 Lookup Table 常数时间完成^[7,14,24]。

算法 1 MiniArray.Access

```

1: 输入逻辑下标  $\text{idx}$ 
2:  $\text{cur} \leftarrow \text{root}$ ,  $\text{rem} \leftarrow \text{idx}$ 
3: while  $\text{cur}$  为内部节点 do
4:   找到子树  $i$  使  $\sum_{t < i} \text{subtree\_size}[t] \leq \text{rem} < \sum_{t \leq i} \text{subtree\_size}[t]$ 
5:    $\text{rem} \leftarrow \text{rem} - \sum_{t < i} \text{subtree\_size}[t]$ 
6:    $\text{cur} \leftarrow \text{cur.child}[i]$ 
7: end while
8:  $\text{pos} \leftarrow \text{CallSelect}(\text{cur.bitmap}, \text{rem})$ 
9: 返回  $\text{cur.data}[\text{pos}]$ 

```

3.1.4 Insert：插入与叶分裂

Insert 在定位叶节点的过程中维护路径栈；在叶内用 Select 找到插入位置，更新 bitmap 并向 data 写入 val 。若叶内有效元素数达到 MAX_LEAF_SIZE ，调用 Split 分裂；最后自底向上对路径栈中每个内部节点的 subtree_size 加一。

3.1.5 Delete：删除与叶合并

Delete 同样沿 subtree_size 定位叶节点，用 Select 找到待删位置，清除 bitmap 对应位并从 data 移除元素。若叶内有效元素过少，调用 Merge 与相邻兄弟叶合并；最后自底向上将路径上 subtree_size 减一。

算法 2 MiniArray.Insert

```
1: 输入 idx, val; 初始化空路径栈 stk
2: cur ← root, rem ← idx
3: while cur 为内部节点 do
4:   stk.push(cur)
5:   找到子树  $i$  使  $\sum_{t < i} subtree\_size[t] \leq rem < \sum_{t \leq i} subtree\_size[t]$ 
6:   rem ← rem -  $\sum_{t < i} subtree\_size[t]$ 
7:   cur ← cur.child[i]
8: end while
9: leaf ← cur
10: pos ← CallSelect(leaf.bitmap, rem)
11: leaf.bitmap.set(pos); leaf.data.insert(pos, val)
12: if leaf.size  $\geq$  MAX_LEAF_SIZE then
13:   CallSplit(leaf)
14: end if
15: for 路径栈中每个内部节点 node do
16:   node.subtree_size[i] += 1
17: end for
```

算法 3 MiniArray.Delete

```
1: 输入 idx; 初始化路径栈 stk
2: 沿 subtree_size 定位至叶 leaf(同 Insert)
3: pos ← CallSelect(leaf.bitmap, rem)
4: leaf.bitmap.unset(pos); leaf.data.remove(pos)
5: if leaf 有效元素过少 (即 leaf.size < MAX_LEAF_SIZE / 2) then
6:   CallMerge(leaf)
7: end if
8: for 路径栈中每个内部节点 node do
9:   node.subtree_size[i] -= 1
10: end for
```

3.1.6 Update: 原地覆写

Update 仅修改已有有效位置的元素值, 不改变 bitmap 中 1 的个数, 也不触发 Split/Merge。定位过程与 Access 相同, 在叶内 Select 后直接 $\text{leaf.data}[\text{pos}] \leftarrow \text{val}$ 。

算法 4 MiniArray.Update

- 1: 输入 idx, val
 - 2: 沿 subtree_size 定位至叶 leaf
 - 3: $\text{pos} \leftarrow \text{CallSelect}(\text{leaf.bitmap}, \text{rem})$
 - 4: $\text{leaf.data}[\text{pos}] \leftarrow \text{val}$
-

3.1.7 Split 与 Merge

Split (叶分裂)。新建叶节点, 以 $\text{leaf.size}/2$ 为界将后半段 bitmap 与 data 迁至新叶, 原叶保留前半段; 在父节点 child 数组中插入新叶指针, 并更新 subtree_size 的分裂计数。

算法 5 MiniArray.Split (叶分裂)

- 1: 输入叶 leaf
 - 2: $\text{new_leaf} \leftarrow$ 新建 LeafNode; $\text{mid} \leftarrow \text{leaf.size}/2$
 - 3: 将 leaf.bitmap 、 leaf.data 后半段搬至 new_leaf
 - 4: leaf 保留前半段
 - 5: 在父节点 parent.child 中插入 new_leaf , 更新 $\text{parent.subtree_size}$
-

Merge (叶合并)。取相邻兄弟叶, 将兄弟的 bitmap 与 data 并入当前叶, 从父节点删去兄弟子树并刷新 subtree_size 。

算法 6 MiniArray.Merge (叶合并)

- 1: 输入叶 leaf ; $\text{sib} \leftarrow$ 相邻兄弟叶; $\text{parent} \leftarrow \text{leaf}$ 的父节点
 - 2: $\text{leaf.bitmap} \mid = \text{sib.bitmap}$; $\text{leaf.data.append}(\text{sib.data})$
 - 3: 从 parent.child 移除 sib , 更新 $\text{parent.subtree_size}$
-

3.1.8 Rank 与 Select

叶内 bitmap 上的 Rank/Select 通过 Lookup Table 实现, 这是紧凑动态结构的标准组件^[3,7,10,24]。

- $\text{Rank}(\text{bitmap}, i)$: 返回 bitmap 前 i 位中 1 的个数，等于 LookupTable 对高段与低段的查表结果之和；
- $\text{Select}(\text{bitmap}, k)$: 返回 bitmap 中第 k 个 1 的位置，由 LookupTable[k] 直接给出。

算法 7 Rank / Select（查表实现）

```

1: function RANK(bmp,  $i$ )
2:   返回 LookupTable[bmp 高段] + LookupTable[bmp 低段]
3: end function
4: function SELECT(bmp,  $k$ )
5:   返回 LookupTable[ $k$ ] ▷ 第  $k$  个 1 的位置
6: end function

```

3.1.9 复杂度与空间性质

Facility 级 MiniArray 的树高为 $O(1)$ ，自根至叶的每层路由仅依赖 subtree_size 比较，叶内 Rank/Select 经 Lookup Table 常数时间完成^[7,24]。因此 Access 与 Update 为最坏情况 $O(1)$ ；Insert 与 Delete 在叶分裂/合并均摊意义下亦为 $O(1)$ ^[3,22]。空间方面，每个内部节点的 F 个子指针各需 $\Theta(\log \log n)$ 比特，叶节点以 bitmap 标记有效位置、以 PackedArray 无间隙存放变长元素，整体辅助位数与 $O(\log^{(k)} n)$ 的全局预算相容^[11,14]。Cubby 内的 A 、 M 、 B 三组 MiniArray 复用同一套接口语义： $A[i]$ 存 router 映射， M 存 bin 空闲图， B 与 storage 槽位一一对应存差分编码——三者在 k-kick 搬移时须原子式同步更新。

3.2 多层级 Cubby 设计

3.2.1 层级属性与初始状态

Facility 内 Cubby 按 tier 分级管理。 j -tiered Cubby 的目标数量为

$$t_j = \Theta\left(\frac{(\log^{(j+1)} n)^2}{(\log^{(j)} n)^2}\right),$$

容量（槽位数）为

$$r_j = \frac{K}{(\log^{(j)} n)^2}.$$

系统初始时，每个 Facility 仅包含一个 1-tiered Cubby，且该 Cubby 即为 tail。此后所有新键插入 tail；tail 满后，旧 tail 被视为一个普通的 1-tiered Cubby 挂入 tier 集合，并新建 tail 承接后续插入，该过程可能触发 tier 向上合并^[2-3,5-6]。

3.2.2 动态调整：合并与拆分

设当前 j -tiered Cubby 的实际个数为 cnt_j 。调度规则如下：

- 当 $\text{cnt}_j \geq 3t_j$ 时，取出 t_j 个 j -tiered Cubby，将其内全部元素导出，合并为一个 $(j+1)$ -tiered Cubby 并重新插入；
- 当 $\text{cnt}_j \leq t_j/2$ 且存在非空的 $(j+1)$ -tiered Cubby 时，取出一个 $(j+1)$ -tiered Cubby，拆成 t_j 个 j -tiered Cubby 并重插元素；
- 无论合并或拆分，均须取出 cubby 内所有元素重新插入，并同步更新对应 Cubby 中的 MiniArray A 、 M 、 B 与 storage，不能仅复制旧槽位内容（因 r_j 、 $|I|$ 与商压缩上下文可能已变）^[3-4,25]。

算法 8 Facility tier 数量调度

```

1: for 每个 tier 层级  $j$  do
2:   计算目标数量  $t_j$ ，统计实际数量  $\text{cnt}_j$ 
3:   if  $\text{cnt}_j \geq 3t_j$  then
4:     取出  $t_j$  个  $j$ -tiered Cubby，合并为一个  $(j+1)$ -tiered Cubby 并重新插入
5:   else if  $\text{cnt}_j \leq t_j/2$  且 tier  $j+1$  非空 then
6:     取出一个  $(j+1)$ -tiered Cubby，拆成  $t_j$  个  $j$ -tiered Cubby 并重新插入
7:   end if
8: end for

```

3.2.3 Cubby 内部组件

每个容量为 $|I|$ 的 Cubby 包含以下五部分：

storage 数组。长度为 $|I|$ ，存放键/value 的商压缩载荷；槽位 i 上的 $\text{storage}[i]$ 对应 $\pi(x)[\log N + 1, \log U]$ 的 payload 部分，完整键恢复依赖 $B[i]$ 与全局上下文。

k-kick 插入逻辑。在 storage 上的按一定的规则动态决定元素实际槽位，并维护每个占用元素的 insert_depth。

MiniArray A 。长度为 $|I|$ 的 local query router 数组； $A[i]$ 是一棵二叉前缀树，管理所有满足 $g_I(x) = i$ 的键。Cubby 容量 $|I|$ 即 router 数组长度。

MiniArray M 。存储各 k -kick 深度 bin 的空闲槽位图，使插入、删除与 ReInsert 能在常数时间内判定 bin 是否含空位、定位空位，而不必扫描整个 Cubby。

MiniArray B 。与 storage 槽位一一对应。当 $\text{storage}[i]$ 存放键 x 的信息时， $B[i]$ 保存

$$\delta = g_I(x) - i, \quad g_I(x) = g^*(x)[\log |I| + 1, \log K],$$

以及 $g^*(x)[\log |I| + 1, \log K]$ 的中间位。配合 Facility 编号 $r = g^*(x)[\log K + 1, \log N]$ 与 payload，可完整恢复 $\pi(x)^{[3,7]}$ 。

3.2.4 插入与删除操作

插入。每次插入均进入当前 Facility 的 tail Cubby。若 tail 仍有空位，则在 tail 内执行 k -kick 插入；若 tail 已满，则新建 tail，并将已满的旧 tail 视作 1-tiered Cubby 挂入 1-tier Cubby 集合，该步骤可能因 $\text{cnt}_1 \geq 3t_1$ 而触发向上合并。

删除与 tail 回填。若删除发生在 tail 之外的其他 Cubby，删除完成后从 tail 中随机选取一个占用元素，经 k -kick 规则回填至刚空出的槽位，并更新 MiniArray A 、 M 、 B 与 storage。若 tail 在回填后为空，则从 1-tiered 层取出一个 Cubby 提升为 tail；该过程可能因 $\text{cnt}_i \leq t_i$ 而触发向下拆分（甚至可能会触发多层）。上述不变量保证：除 tail 外，各 Cubby 尽量维持满载，局部波动由 tail 与 tier 调度共同吸收^[3,15-16]。

算法 9 tail 维护与删除回填

- 1: 插入：写入 tail；若 tail 满则旧 tail 归入 tier 0 并新建 tail
 - 2: 删除（非 tail）：释放被删槽位；从 tail 随机取元素 y 回填
 - 3: 对 y 执行 k -kick 写入，更新 A, M, B
 - 4: **if** tail 为空 **then**
 - 5: 从 1-tiered 层提升一个 Cubby 为 tail；必要时触发 tier 拆分
 - 6: **end if**
-

3.3 k -kick 层级插入算法

为了方便，做以下约定：Cubby 内算法中的 x 均指 $\pi(x)$ （置换后的值），而非原始键；符号仍记为 x ，这里不妨设 Cubby 的容量为 $|I|$ 。

3.3.1 层级 bin 定义

k-kick 不对 storage 做物理分层，仅在逻辑上将 Cubby 划分为 $k + 1$ 个深度， k 为常数（通常 $k \in [3, 7]$ ，基准取 4），定义不同层次的 bin：

- 深度 0：唯一 bin 覆盖整个 Cubby，大小 $s_0 = K = \text{polylog } n$ ；
- 深度 1：将深度 0 的 bin 均匀切分为许多大小 $s_1 = \Theta((\log K)^6)$ 的子 bin；
- ...
- 深度 i ($i \geq 1$)：bin 大小 $s_i = \Theta((\log^{(i)} K)^6)$ ，由上一层 bin 继续均匀切分得到。

函数 $\text{Bin}(x, i)$ 返回键 x 在深度 i 所属 bin 的槽位区间：深度 0 时返回 $[0, |I| - 1]$ ；深度 $i \geq 1$ 时在 $\text{Bin}(x, i - 1)$ 内按 s_i 均匀切分，并根据 x 的哈希比特选定其中一个子区间^[3,20]。

3.3.2 偏好序列

对键 x 定义偏好序列

$$g_0(x) \rightarrow g_1(x) \rightarrow \cdots \rightarrow g_k(x).$$

其中 $g_0(x) = \text{Bin}(x, 0)$ 为整个 Cubby； $g_{i+1}(x)$ 为 $g_i(x)$ 经均匀切分后 x 所落子 bin。于是 $g_{i+1}(x)$ 一定是 $g_i(x)$ 的子 bin；沿路径自顶向下，每一层均为上一层的随机子区间。自底向上回溯 $g_k, g_{k-1}, \dots, g_0$ 即可得到全部祖先 bin。

算法 10 PreferenceSequence: 偏好序列

```

1: function PREFERENCESEQUENCE( $x$ )
2:    $g[0] \leftarrow \text{BIN}(x, 0)$ 
3:   for  $i = 1$  to  $k$  do
4:      $g[i] \leftarrow g[i - 1]$  内均匀切分后、 $x$  所属的子 bin
5:   end for
6:   返回  $g[0..k]$ 
7: end function

```

3.3.3 探测位置序列

$h_j(x)$ 表示 x 在 Cubby storage 中的第 j 个候选槽位。序列按先深后浅顺序构造：依次遍历 $g_k(x), g_{k-1}(x), \dots, g_0(x)$ ，并在每个 bin 内按槽位顺序枚举全部位置。

对深度 i 的 bin $g_i(x)$ ，设

$$\text{base}_i = s_k + s_{k-1} + \dots + s_{i+1},$$

则对任意 $j \in [0, s_i - 1]$ ，全局探测序列中对应项为

$$h_{\text{base}_i+j}(x) = g_i(x) \text{ 的第 } (j+1) \text{ 个槽位.}$$

注意文档中下标公式 $h_{(k+1-i) \cdot s_i + j}(x)$ 与 base_i 表述等价：更深层的 bin 在序列中排在更前。序列总长度约为 $\sum_{i=0}^k s_i$ ；查询阶段并不线性扫描该序列，而由 Local Query Router 直接给出实际使用的下标 $j^{[3,18]}$ 。

算法 11 ProbeSequence: 探测位置序列（先深后浅）

```

1: function PROBESEQUENCE( $x$ )
2:    $g \leftarrow \text{PREFERENCESEQUENCE}(x)$ ;  $\text{seq} \leftarrow []$ 
3:   for  $i = k$  downto 0 do
4:      $\text{bin} \leftarrow g[i]$ 
5:     for  $\text{pos}$  从  $\text{bin.start}$  到  $\text{bin.end}$  do
6:        $\text{seq.append}(\text{pos})$ 
7:     end for
8:   end for
9:   返回  $\text{seq}$ 
10: end function

```

$\triangleright h_1(x), h_2(x), \dots$ 依次为 $\text{seq}[0], \text{seq}[1], \dots$

3.3.4 bin 饱和与最大可用深度

在深度 d 下，称 bin 饱和，当且仅当 bin 已满，且其中每个元素 y 均满足 $y.\text{insert_depth} \geq d$ 。bin 未饱和时不饱和；bin 已满但存在 $y.\text{insert_depth} < d$ 的元素时，亦不视为饱和。

GetMaxUsableDepth(x) 自 k 向 0 扫描偏好序列，返回最深的饱和 bin 的深度；若全部饱和则返回 0(说明已满)。

算法 12 IsSaturated 与 GetMaxUsableDepth

```
1: function ISSATURATED(bin,  $d$ )
2:   if bin 未滿 then
3:     返回 false
4:   end if
5:   for bin 中每个元素  $y$  do
6:     if  $y.\text{insert\_depth} < d$  then
7:       返回 false
8:     end if
9:   end for
10:  返回 true
11: end function
12: function GETMAXUSABLEDEPTH( $x$ )
13:   $g \leftarrow \text{PREFERENCESEQUENCE}(x)$ 
14:  for  $d = k$  downto 0 do
15:    if not ISSATURATED( $g[d]$ ,  $d$ ) then
16:      返回  $d$ 
17:    end if
18:  end for
19:  返回 0
20: end function
```

3.3.5 随机深度选择与 InsertKick

插入前先取随机哈希 $s(x) \in [0, k]$ ，再令

$$\text{depth} = \min(\text{GETMAXUSABLEDEPTH}(x), s(x)).$$

在目标 bin $g_{\text{depth}}(x)$ 中：若有空位，则写入 x 、记录 $x.\text{insert_depth} = \text{depth}$ ，并在 $A[g_I(x)]$ 中登记探测下标 j ；若无空位，则踢出 bin 内 insert_depth 最小的元素 y ，将 x 写入被踢槽位，再对 y 调用 ReInsert。整条踢出链至多触发 k 次，与 $O(k)$ 交换上界一致^[3]。

3.3.6 ReInsert：被踢元素向上安置

ReInsert(y) 自 $y.\text{insert_depth}-1$ 向上逐层检查 Bin(y, d)：若该 bin 在深度 d 下不饱和且有空位，则将 y 写入空位、令 $y.\text{insert_depth} = d$ 并返回。MiniArray M 在此过程中的空位判定与定位均为常数时间。理论保证：在参数满足 Bender 等人^[3] 的设定时，至多 k 次 kick 后一定能安置被踢元素。

算法 13 InsertKick: k-kick 插入

```
1: 输入  $x$  (已为  $\pi(x)$ )
2:  $depth\_max \leftarrow \text{GETMAXUSABLEDEPTH}(x)$ 
3:  $s_x \leftarrow \text{RANDOMHASH}(x) \bmod (k + 1)$ 
4:  $depth \leftarrow \min(depth\_max, s_x)$ 
5:  $g \leftarrow \text{PREFERENCESEQUENCE}(x)$ ;  $target \leftarrow g[depth]$ 
6: if  $target$  有空位 (由  $M$  判定) then
7:   在任意空位写入  $x$ ;  $x.insert\_depth \leftarrow depth$ 
8:   更新  $A, M, B$ ; 返回成功
9: else
10:   $y \leftarrow target$  中  $insert\_depth$  最小元素;  $evict\_pos \leftarrow y$  的位置
11:  从  $evict\_pos$  移除  $y$ ; 将  $x$  写入  $evict\_pos$ 
12:   $x.insert\_depth \leftarrow depth$ 
13:   $\text{REINSERT}(y)$ 
14: end if
```

算法 14 ReInsert: 被踢元素向上找空位

```
1: function  $\text{REINSERT}(y)$ 
2:   for  $d = y.insert\_depth - 1$  downto 0 do
3:      $bin \leftarrow \text{BIN}(y, d)$ 
4:     if not  $\text{ISATURATED}(bin, d)$  and  $bin$  有空位 then
5:       将  $y$  写入空位;  $y.insert\_depth \leftarrow d$ 
6:       更新  $A, M, B$ ; 返回
7:     end if
8:   end for
9: end function
```

3.3.7 复杂度与正确性

单次 InsertKick 至多触发一条长度为 k 的踢出链：每次踢出仅调用一次 ReInsert，而被踢元素的 insert_depth 严格下降，故链长受 k 约束，与 Bender 等人^[3]的 $O(k)$ 交换成本一致。

GetMaxUsableDepth 自 k 向 0 扫描，返回最深的饱和 bin；随机深度 $\text{depth} = \min(\text{depth_max}, s(x))$ 在理论上保证：若存在可写 bin，则插入不会无谓触发深层踢出，同时使不同深度的 bin 负载趋于均衡^[17]。

空位判定与定位均通过 MiniArray M 在 bin 粒度完成，避免 $O(|I|)$ 扫描；ReInsert 自 $y.\text{insert_depth}-1$ 向上逐层检查，每层仅访问 M 与局部 bin 状态，均摊常数时间。

元素每次写入或搬移，须同时更新 storage、insert_depth、 B 、 $A[g_I(x)]$ 中的 $\pi(x) \mapsto j$ 以及 M 的空闲位图。踢出时先从 A 删除被踢键的旧映射，安置后再插入新 j ；该同步规则是 Local Query Router 保持 $O(1)$ 查询的前提。

3.4 Local Query Router

3.4.1 设计目标

偏好序列与探测位置序列 $h_1(x), h_2(x), \dots$ 在插入分析中完整定义了候选槽位顺序。然而 k-kick 插入会通过踢出动态挪动元素：元素实际位置一般不等于首选深度 $g_k(x)$ ，也不等于 $h_1(x)$ 。若查询仍沿 $h_1, h_2, \dots$ 线性扫描，时间复杂度将与探测序列长度同阶，无法达到 $O(1)$ 。因此必须为每个键保存精准映射

$$\pi(x) \longrightarrow j,$$

其中 j 为探测序列中的下标，查询时直接访问 $h_j(x)$ ，再用 B 与 payload 校验^[3,7]。

3.4.2 映射规则

Cubby 内定义局部首选槽位号（与 k-kick 偏好序列无关）：

$$g_I(x) = g^*(x)[1, \log |I|] \in [0, |I| - 1].$$

所有满足 $g_I(x) = i$ 的键由第 i 号 local query router $A[i]$ 管理。查询时先计算 $i = g_I(x)$ ，再在 $A[i]$ 中查询 $\pi(x)$ 得到探测序列下标 j （若不存在则未命中）；随后访问 storage 中 $h_j(x)$ 对应槽位，读取 payload 与 B ，重建 $g^*(x)$ 并校验是否命中。

3.4.3 二叉前缀树存储与接口

每个 $A[i]$ 实现为一棵二叉前缀树；对外键为 $\pi(x)$ 的比特串。接口为：

- $\text{insert}(\text{key}, j)$: $j \leq \log K$ ，仅需 $O(\log \log n)$ 比特存储；
- $\text{query}(\text{key})$: 返回 j 或 “不存在”；
- $\text{delete}(\text{key})$: 删除映射。

树中不存完整键，仅存区分键的最短前缀；叶节点存 j_value 。

算法 15 Local Query Router.insert

```

1: function INSERT( $key, j$ )
2:    $cur \leftarrow root$ 
3:   for  $key$  的每一位  $bit$  do
4:     if  $bit = 0$  then
5:       if  $cur.child0$  为空 then 创建子节点
6:       end if
7:        $cur \leftarrow cur.child0$ 
8:     else
9:       if  $cur.child1$  为空 then 创建子节点
10:      end if
11:       $cur \leftarrow cur.child1$ 
12:    end if
13:  end for
14:   $cur.is\_leaf \leftarrow true$ ;  $cur.j\_value \leftarrow j$ 
15: end function

```

3.4.4 复杂度与 Patricia 压缩

对键长 $\log U = \Theta(\log n)$ 的比特串，query 与 insert 沿树向下至多 $\log U$ 步；Patricia 风格的前缀压缩^[7,13,26] 将仅有一位不同的键合并为同一压缩边，使实测 router 深度（router_steps_max）远小于键长上界。每条映射存 $j \leq \log K = O(\log \log n)$ 比特，与整体 $O(\log^{(k)} n)$ 空间目标一致^[3-4]。

算法 16 Local Query Router.query

```
1: function QUERY(key)
2:   cur  $\leftarrow$  root
3:   for key 的每一位 bit do
4:     if bit = 0 then
5:       if cur.child0 为空 then 返回 NOT_FOUND
6:       end if
7:       cur  $\leftarrow$  cur.child0
8:     else
9:       if cur.child1 为空 then 返回 NOT_FOUND
10:      end if
11:      cur  $\leftarrow$  cur.child1
12:    end if
13:  end for
14:  if cur.is_leaf then 返回 cur.j_value
15:  else 返回 NOT_FOUND
16:  end if
17: end function
```

算法 17 Local Query Router.delete

```
1: function DELETE(key)
2:   沿 key 的比特路径找到叶节点
3:   将叶标记为非叶 (is_leaf  $\leftarrow$  false), 清除 j_value
4: end function
```

动态维护。k-kick 踢出链中，被搬移键须 delete 旧映射再 insert 新 j ；删除操作清空叶 j_value 但不破坏共享前缀路径上的内部节点，均摊开销与 $A[i]$ 所管理的局部键数相关。在 $K = \text{polylog } n$ 且 Cubby 容量 $|I| \leq K$ 的设定下，单个 $A[i]$ 的规模受控，router 更新与查询均摊为常数时间^[2-3]。

3.5 插入、查询与删除的统一语义

插入。计算 $g^*(x) = \pi(x)[1, \log N]$ 与 Facility 编号 r ；在对应 Facility 的 tail Cubby 内执行 InsertKick（算法 13），写入 storage、更新 M 与 $B[i]$ ，并在 $A[g_I(x)]$ 中 $\text{insert}(\pi(x), j)$ 。tail 满则挂入 tier 0 并新建 tail，必要时按算法 8 触发合并^[3,17]。

查询。计算 $g^*(x)$ 定位 Facility 与 Cubby；令 $i = g_I(x)$ ，在 $A[i]$ 中 $\text{query}(\pi(x))$ 得 j ；访问 $h_j(x)$ ，用 payload、 B 、 r 重建 $g^*(x)$ 并与 $\pi(x)$ 比较。该路径仅含常数次 MiniArray 与 router 访问，查询时间为 $O(1)$ ^[3]。

删除。定位元素所在 Cubby 与槽位，清空 storage，更新 M 、 B 与 $A[g_I(x)].\text{delete}(\pi(x))$ 。若删除发生在非 tail Cubby，按算法 9 从 tail 随机抽取元素回填；回填写入同样走 k-kick 与 router 更新路径，以恢复各 bin 的饱和与深度不变量。

表级扩缩容时， N 、 K 、 r_j 与商压缩上下文变化，须先恢复原始键再按新参数重插，而不能直接复制旧槽位的 payload 与 B ^[2-3]。

3.6 本章小结

本章完整展开了分层哈希表的结构与算法：Facility 级 MiniArray 通过 B 树、Rank/Select 与 Split/Merge 实现均摊常数时间的变长元数据维护^[22,24]；多 tier Cubby 与 tail 机制定义 tier 调度与删除回填不变量^[5-6]；k-kick 在逻辑 bin 上通过 PreferenceSequence、饱和判定、InsertKick 与 ReInsert 实现 $O(k)$ 交换插入，并与 MiniArray M 协同完成空位判定^[3,8]；Local Query Router 通过 $A[i]$ 上的前缀树将查询降为 $O(1)$ ，Patricia 压缩控制 router 深度^[7,26]。四路同步更新规则贯穿插入、踢出、删除回填与扩缩容重插。第四章在此完整实现之上报告宏基准与多预设扫描下的延迟、踢出与扩缩容等性能测量结果。

第四章 系统性能实验

4.1 实验目的与环境

本章在第二、三章给出的结构与参数设定基础上，对已实现系统进行性能测试。实验不经过 HTTP 与 MySQL 持久化路径，仅在内存态上测量插入、查询、删除等操作的延迟与结构指标，用以检验第三章中三条设计主线是否在实测中有所体现：

- k-kick 插入过程中踢出强度与单次搬移上界；
- Local Query Router 是否保持较短的查询路径；
- MiniArray 与路由维护带来的元数据开销是否随负载增长。

全部实验于 2026 年 5 月 30 日在本机完成，默认随机种子为 1（部分补充实验另做种子对比）。

测试环境为作者开发机，使用项目自带的两个命令程序：otsh_perf 用于单一配置下的宏基准；otsh_experiment 用于多组预设参数扫描及补充实验。二者均输出各阶段 p50/p99 延迟、踢出累计值、扩缩容与 tier 重建次数等指标。需要说明的是，本文未与 IcebergHT^[6]、PaCHash^[14] 等外部实现进行同机对比，以下结果反映的是本系统在既定参数下的表现，并与第三章机制对照说明^[2-3]。

4.2 实验方案

4.2.1 测试程序与工作量

表 4-1 概括各轮实验的用途与工作量设置。前两行为基础实验，后三行为在初步结果基础上增加的补充实验，用于观察更长插入序列、随机种子变化以及元数据开销等问题。

规模模式中， $n_{hint} = 10^6$ 的插入次数取 20 万次，约为参数配置文件中 40

表 4-1 实验方案一览

名称	程序与模式	主要工作量
宏基准	otsh_perf	插入 10^4 键；随机命中查询 5×10^4 次；删半键；混合查询
参数扫描	otsh_experiment（快速模式）	12 组预设各 2000 次插入、2000 次命中查询、500 次删除
规模扩展	otsh_experiment（规模模式）	$n_hint = 10^4/10^5/10^6$ 下分别约 4k / 40k / 200k 次插入
种子对比	otsh_experiment（快速模式）	baseline 预设下 seed 取 1、2、3、5、42 各跑一轮
元数据对照	规模模式 + 快速模式	终态统计逻辑 meta_bits 与槽位占用粗算值

万次的一半，以便在单机条件下控制实验时间；每轮结束后会等待后台 rebuild 任务执行完毕再统计指标。

4.2.2 评价指标

实验主要关注以下三类指标：

- 延迟：以 p50、p99 纳秒延迟及吞吐为主，p50 反映典型路径，p99 反映扩容等偶发长尾；
- 结构行为：包括 kick/ins（平均每次插入的踢出次数）、insert_moved_max（单次插入搬移上界）、router_steps_max（查询路由步数上界）、resize 与 rebuild 事件次数等；
- 空间相关统计：meta_bits_total 为逻辑编码位数累计；另在终态遍历 cubby，按占用槽位与容量、cubby 个数粗算物理槽位占用（记为 phys_est），用于与逻辑统计对照。

4.3 宏基准实验

4.3.1 实验配置

宏基准在 $table_n_hint=10^6$ 、 $n_insert=10^4$ 、 $n_rand_queries=5 \times 10^4$ 下运行。初始化耗时 113.3 ms；删半键后终态元素数 $n = 5000$ ，表规模 $N = 16384$ ，Facility 规模 $K = 2048$ ，Facility 数为 8。

4.3.2 各阶段延迟

表 4-2 给出各操作阶段的吞吐与 p50 延迟。可以看出，三类查询的 p50 均在 $4\text{--}5\ \mu\text{s}$ 附近，与第三章所述“经 Local Query Router 定位后再做商压缩校验”的短路径相符^[3,26]。插入 p50 为 $19.5\ \mu\text{s}$ ，而平均延迟为 $74.8\ \mu\text{s}$ ，约为 p50 的 3.8 倍，说明少数插入触发了扩缩容或较长踢出链，拉高了均值。删除 p50 为 $27.2\ \mu\text{s}$ ，高于查询、低于插入均值，符合释放槽位、更新路由及非 tail 回填等操作的预期^[16]。

表 4-2 宏基准各阶段延迟 ($n_{\text{insert}}=10^4$ ，删半后 $n = 5000$)

阶段	操作数	p50 (ns)	p50 (μs)	吞吐 (ops/s)
插入	10000	19500	19.5	13348
顺序命中查询	10000	4200	4.2	229054
随机命中查询	50000	4700	4.7	202494
删半键	5000	27200	27.2	33805
混合查询	25000	4400	4.4	247079

4.3.3 终态结构指标

表 4-3 汇总该次运行的累计结构指标。 insert_moved_max 与 delete_moved_max 均为 1，未超过第三章 k-kick 在 $k = 4$ 时的 $O(k)$ 交换上界，说明在当前以 tail 为主的负载下，多数插入在较浅层 bin 内即可完成。 router_steps_max 为 8，与 Patricia 压缩路由深度一致。 resize_start 与 resize_finish 均为 12，表明双表扩缩容均已收尾； rebuild_up/down 为 0，说明在本轮工作量下尚未触发 tier 合并或拆分。

表 4-3 宏基准终态累计结构指标

指标	测量值
$\text{insert_moved_total} / n_{\text{insert}}$	$725 / 10000 = 0.073$
insert_moved_max	1
$\text{delete_moved_total}$	1995
delete_moved_max	1
router_steps_max	8
$\text{avg_meta_bits_per_insert}$	$\approx 1.01 \times 10^5$
$\text{rebuild_up} / \text{rebuild_down}$	0 / 0
$\text{resize_start} / \text{resize_finish}$	12 / 12
终态 ($n, N, K, \text{facilities}$)	(5000, 16384, 2048, 8)

其中 meta_bits_total 统计的是逻辑编码位数累计，反映 MiniArray 与路由维护随插入单调增加的趋势^[7,14]，其含义将在后文元数据对照实验中进一步说明。

4.4 多预设参数扫描

在快速模式下，对系统重构方案中的 12 组预设各执行 2000 次插入、2000 次命中查询与 500 次删除。表 4-4 列出派生参数 (N, K, k) 及主要测量结果，各组运行均通过正确性检查。

表 4-4 12 组预设快速扫描结果 (2026-05-30, 每组 2000 插入)

预设	N	K	k	kick total	kick/ins	resize events	ins p50 (μs)	qry p50 (μs)
baseline	2^{20}	4096	4	140	0.070	20	20.3	4.4
n_1e4	2^{14}	2048	4	200	0.100	48	16.7	4.1
n_1e5	2^{17}	4096	4	220	0.110	74	16.6	3.7
n_1e6	2^{20}	4096	4	175	0.088	94	18.9	3.7
n_1e7	2^{24}	4096	4	26	0.013	95	109.9	10.1
K_log2n	2^{20}	256	4	40	0.020	96	26.8	9.3
K_log4n	2^{20}	4096	4	484	0.242	116	23.3	4.5
k3	2^{19}	4096	3	96	0.048	138	17.3	3.8
k5	2^{19}	4096	5	530	0.265	160	18.5	4.7
k7	2^{19}	4096	7	1316	0.658	182	21.3	3.5
lf_high	2^{20}	4096	4	182	0.091	202	19.5	4.0
lf_low	2^{20}	4096	4	171	0.086	222	18.9	3.9

表中 kick/ins 为 kick_total 除以插入次数；resize_events 为 resize_start 与 resize_finish 之和。

4.5 补充实验

快速扫描每组仅 2000 次插入，尚不足以观察长跑负载下的行为差异。为此增加以下三组补充实验。

4.5.1 较大规模插入

在规模模式下，对 $n_hint \in \{10^4, 10^5, 10^6\}$ 分别执行 4000、40000、200000 次插入，再按比例删除部分键并进行全命中查询。表 4-5 汇总结果。

与表 4-4 对比， n_1e6 在 20 万次插入下 kick/ins 由 0.088 升至 0.269，插入 p50 由 $18.9 \mu s$ 升至 $33.3 \mu s$ ，查询 p50 由 $3.7 \mu s$ 升至 $5.3 \mu s$ ，说明负载加深后踢出与局部访问开销有所累积；insert_moved_max 仍为 1，单次搬移上界仍未突破 k 。三组结果中 rebuild_up/down 均为 0，终态 cubby 数虽随规模增大（如 n_1e6 达

表 4-5 较大规模插入实验结果 (seed=1, 2026-05-30)

预设	插入	终态 n	cubbies	ins p50 (μ s)	ins p99 (μ s)	qry p50 (μ s)	kick/ins	resize events
n_1e4	4000	2000	290	16.7	74.4	4.3	0.314	32
n_1e5	40000	20000	1698	21.8	75.0	4.2	0.332	66
n_1e6	200000	100000	12318	33.3	108.4	5.3	0.269	98

12318 个), 但分散在多个 Facility 中, 尚未使任一 Facility 内 tier- j 的 cubby 计数稳定超过 $3t_j$ 阈值。

4.5.2 随机种子对比

为考察结果是否过度依赖某一组随机键, 在 baseline 预设 ($N = 2^{20}$, $K = 4096$) 与快速工作量下, 对 seed 取 1、2、3、5、42 各运行一轮, 结果见表 4-6。

表 4-6 baseline 预设下不同随机种子的测量结果 (2000 次插入)

seed	kick/ins	ins p50 (μ s)	ins p99 (μ s)	qry p50 (μ s)
1	0.093	19.2	2064	3.7
2	0.080	19.1	2102	3.8
3	0.085	19.0	2149	3.8
5	0.078	19.9	1964	4.0
42	0.091	19.6	2150	4.1

五组种子下, kick/ins 介于 0.078–0.093, 插入与查询 p50 的相对波动小于 6%, 说明中位数延迟与踢出强度对键序列不十分敏感。插入 p99 在 1.96–2.15 ms 之间波动, 主要由偶发扩缩容引起, 复现时除报告 p50 外亦需关注尾延迟。

4.5.3 逻辑元数据与槽位占用对照

逻辑 meta_bits 在每次插入、删除时累加 Facility 级路由与 MiniArray 编码位数, 反映维护过程中的编码工作量; phys_est 则按终态槽位占用与 cubby 数量粗算物理骨架规模。表 4-7 给出二者比值。

比值随规模增大而升高, 表明路由与 Rank/Select 维护在逻辑统计中占比较大, 不能仅用槽位数量解释整体开销。该粗算尚未计入进程实际驻留内存 (RSS), 后续工作可在此基础上增加物理内存测量。

表 4-7 逻辑 meta_bits 与槽位占用粗算值对照 (seed=1)

预设	插入次数	meta_total	phys_est	比值
n_1e4 (规模模式)	4000	2.74×10^8	2.18×10^5	$\approx 1.3 \times 10^3$
n_1e5 (规模模式)	40000	7.01×10^9	1.93×10^6	$\approx 3.6 \times 10^3$
n_1e6 (规模模式)	200000	5.64×10^{10}	1.07×10^7	$\approx 5.3 \times 10^3$
baseline (快速模式)	2000	7.35×10^7	1.82×10^5	$\approx 4.0 \times 10^2$

4.6 实验结果分析

本节综合宏基准、参数扫描与补充实验，从规模、结构参数与系统行为三方面归纳规律。

4.6.1 数据规模与查询延迟

在 $n_hint \in \{10^4, 10^5, 10^6\}$ 的快速扫描中，命中查询 p50 稳定在 3.7–4.1 μs ，插入 p50 为 16.6–20.3 μs ，说明在中等表规模下查询路径未随 Facility 数量失控。当 $n_hint = 10^7$ ($N = 2^{24}$) 时，插入 p50 升至 109.9 μs ，查询 p50 升至 10.1 μs ，Facility 数增至 4096 后，局部索引与缓存局部性成本明显上升，但查询仍低于 20 μs ^[1-2]。规模扩展实验进一步表明，n_1e6 在 20 万次插入后查询 p50 为 5.3 μs ，较快速扫描略有上升，仍属可接受范围。

4.6.2 Facility 规模参数 K 的影响

对比 K_log2n ($K = 256$) 与 baseline ($K = 4096$)：在相同 $N = 2^{20}$ 下，更小 K 使 kick/ins 由 0.070 降至 0.020，但查询 p50 由 4.4 μs 升至 9.3 μs ，插入 p50 亦有所上升。这说明 K 减小后单 Facility 内竞争减弱，分区变细却带来更高的路由与索引成本^[3-4]。 K_log4n 与 baseline 的 K 相同，但 kick/ins 升至 0.242，表明 K 的派生指数仍会影响 Cubby 容量与踢出强度。

4.6.3 k-kick 深度 k 的影响

在 $N = 2^{19}$ 、 $K = 4096$ 下， k 从 3 增至 7 时，kick/ins 由 0.048 单调升至 0.658，插入 p50 由 17.3 μs 升至 21.3 μs ，查询 p50 仍在 3.5–4.7 μs 之间。这与文献中“更大 k 允许更深踢出、插入期交换增多，查询仍近似常数时间”的权衡一致^[3,7,26]。

4.6.4 负载因子与扩缩容

lf_high ($\alpha = 0.95$) 与 lf_low ($\alpha = 0.75$) 的 $kick/ins$ 接近 (0.091 与 0.086), 插入与查询 p50 相差不足 $1\ \mu s$, 但 $resize$ 事件明显多于 $baseline$ (202/222 对 20)。宏基准中插入均值远高于 p50, 亦与插入阶段多次扩缩容有关^[2,25]。

4.6.5 tier 重建与双表迁移

全部实验中 $rebuild_up/down$ 均为 0: 快速扫描、宏基准、规模扩展与种子对比均未触发 $tier$ 合并或拆分; 在 n_1e5 上追加约 5 万次插入的压力测试中, 终态 $cubby$ 数达 3085, $rebuild$ 计数仍为 0。扩缩容方面, $resize_start$ 与 $resize_finish$ 成对出现, 说明双表渐进迁移按第三章设计正常工作^[2-3]。 $tier$ 调度未触发, 是本章实验的主要留白之一。

4.7 本章小结

本章通过宏基准、12 组预设扫描及三组补充实验, 对本系统进行了较完整的性能评估。主要结论如下:

- 命中查询 p50 多在 $4\text{--}5\ \mu s$, 插入 p50 多在 $10\text{--}20\ \mu s$, 单次搬移上界未超过 $O(k)$;
- 在中等 n 下查询延迟较稳定, $n = 10^7$ 时延迟明显上升; k 与 K 对 $kick/ins$ 影响显著;
- 20 万次插入的规模实验表明, 长跑负载会抬高 $kick/ins$ 与查询延迟, 但 $insert_moved_max$ 仍为 1;
- 五组随机种子下 p50 波动小于 6%, 结果具有一定可复现性;
- 逻辑 $meta_bits$ 远高于槽位粗算值, 说明元数据维护开销需单独度量。

尚未覆盖的方面包括 $tier$ 重建触发条件、与外部实现的对比, 以及进程级物理内存测量, 将在第五章简要讨论。

第五章 总结与展望

5.1 工作总结

本文以 Bender 等人提出的 OTSH 框架^[3]为理论基础，完成了分层哈希表的结构与算法设计，并据此实现原型系统、开展性能实验。

在设计层面，第二、三章给出了 Facility-Cubby-Slot 分层组织、Facility 级 MiniArray、k-kick 层级插入、Local Query Router 与商压缩等模块的完整方案。MiniArray 用 B 树与 Rank/Select 维护变长元数据；k-kick 在逻辑 bin 上以 $O(k)$ 次交换完成插入；Local Query Router 通过前缀树将查询映射到目标槽位；各模块经 storage、 B 、 M 与 router 的同步更新保持一致性。

在实验层面，第四章在内存态下对系统进行了宏基准测试、12 组参数扫描以及规模扩展、种子对比、元数据对照等补充实验。主要结果可概括如下：

- 命中查询 p50 多在 $4\text{--}5\ \mu\text{s}$ ，明显快于插入与删除，与查询走短路径的设计目标一致；
- 在 $n_{\text{hint}} \leq 10^6$ 时查询延迟较稳定， k 从 3 增至 7 时 kick/ins 由 0.048 升至 0.658，体现了 k 对插入代价的影响；
- 20 万次插入的规模实验表明，长跑负载会抬高 kick/ins 与延迟，但单次搬移上界 insert_moved_max 仍为 1；
- 五组随机种子下 p50 波动小于 6%，说明典型性能对键序列不十分敏感；
- 逻辑 meta_bits 远高于槽位占用粗算值，提示元数据维护在开销中占比较大^[3-4,7,14]。

综上，本文完成了从理论结构到可运行原型的设计与实现，实验结果与第三章的主要机制预期大体一致，达到了本科毕业设计“有方案、有实现、有测试、有分析”的要求。

5.2 不足与局限

受时间与实验条件限制，本文仍存在以下不足：

- tier 重建尚未在实验中触发。尽管已将插入规模提高到数万至二十万次，`rebuild_up/down` 计数仍为 0。`cubby` 总数随负载增加，但分散在各 Facility 中，尚未使单一 Facility 内 tier 计数稳定超过 $3t_j$ 阈值，因此第三章 tier 调度算法的实际收益尚无法在数据中体现。
- 部分指标含义不同，不宜直接对比。`kick_total` 统计的是踢出链累计长度，`insert_moved_max` 反映的是单次操作上界。例如 k7 预设下 `kick/ins` 可达 0.658，而长跑实验中 `insert_moved_max` 仍为 1，解读实验结果时需区分平均踢出强度与单次最坏搬移^[3,8]。
- 缺少与同类系统的同机对比。本文未将实现与 IcebergHT^[6]、PaCHash^[14] 等放在同一环境下比较延迟与空间，因而难以判断本原型在工程上的相对位置。元数据实验亦仅做了逻辑统计与槽位粗算，未测量进程实际占用内存。
- 大规模与尾延迟问题仍较突出。当 $n_{hint} = 10^7$ 时，插入 p50 达 $109.9 \mu s$ ； n_{1e6} 在 20 万次插入下插入 p99 达 $108.4 \mu s$ ，扩缩容等偶发操作仍是尾延迟的主要来源^[5-6]。

5.3 展望

未来将从以下几方面展开：

- 构造使 `cubby` 集中于少数 Facility 的偏斜负载，或在校准环境中调节 tier 阈值，以便观察 `rebuild` 触发前后 `cntj` 与 t_j 的变化；
- 按参数配置文件跑满更大规模插入（如 40 万次），并区分扩缩容与 tier 重建对 p99 的贡献^[17,21]；
- 在现有逻辑 `meta` 统计之外，增加进程 RSS 或分配器层面的物理内存测量，更准确地评估 Packed Leaf、路由树与 B 编码的开销^[10,14]；
- 选取一至两个紧凑哈希开源实现进行同机对比，从 `kick/ins`、路由深度与查询延迟等维度检验本方案的实际竞争力^[11-12]。

参考文献

- [1] KNUTH D E. The Art of Computer Programming, Volume 3: Sorting and Searching[M]. Reading, MA: Addison-Wesley, 1973.
- [2] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. Modern Hashing Made Simple[C/OL]//Proceedings of the 2024 SIAM Symposium on Simplicity in Algorithms (SOSA 2024). SIAM, 2024: 363-373. DOI: 10.1137/1.9781611977936.33.
- [3] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. On the Optimal Time/Space Tradeoff for Hash Tables[C/OL]//Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing. Rome, Italy: ACM, 2022: 1284-1297. DOI: 10.1145/3519935.3519969.
- [4] KUSZMAUL W, WALZER S. Space Lower Bounds for Dynamic Filters and Value-Dynamic Retrieval[C/OL]//Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC 2024). ACM, 2024: 1153-1164. DOI: 10.1145/3618260.3649649.
- [5] BENDER M A, CONWAY A, FARACH-COLTON M, et al. Iceberg Hashing: Optimizing Many Hash-Table Criteria at Once[J/OL]. Journal of the ACM, 2023, 70(6): 40:1-40:51. DOI: 10.1145/3625817.
- [6] PANDEY P, BENDER M A, CONWAY A, et al. IcebergHT: High Performance Hash Tables Through Stability and Low Associativity[C/OL]//Proceedings of the ACM on Management of Data: Proceedings of the ACM SIGMOD International Conference on Management of Data: vol. 1: 1. Seattle, WA, USA: ACM, 2023: 47:1-47:26. DOI: 10.1145/3588727.
- [7] DIETZFELBINGER M, PAGH R. Succinct Data Structures for Retrieval and Approximate Membership[C/OL]//Lecture Notes in Computer Science: Pro-

- ceedings of the 35th International Colloquium on Automata, Languages, and Programming (ICALP 2008): vol. 5125. Springer, 2008: 385-396. DOI: 10.1007/11523468_30.
- [8] PAGH R, RODLER F F. Cuckoo Hashing[J/OL]. Journal of Algorithms, 2004, 51(2): 122-144. DOI: 10.1016/j.jalgor.2004.03.002.
 - [9] LEHMANN H P, SANDERS P, WALZER S. SicHash – Small Irregular Cuckoo Tables for Perfect Hashing[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2023). SIAM, 2023: 176-189. DOI: 10.1137/1.9781611977561.ch15.
 - [10] LEHMANN H P, MÜLLER T, PAGH R, et al. Modern Minimal Perfect Hashing: A Survey[J/OL]. ACM Computing Surveys, 2026, 58(10): 251:1-251:36. DOI: 10.1145/3797036.
 - [11] LEHMANN H P, SANDERS P, WALZER S. ShockHash: Towards Optimal-Space Minimal Perfect Hashing Beyond Brute-Force[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2024). SIAM, 2024: 194-206. DOI: 10.1137/1.9781611977929.15.
 - [12] HERMANN S, LEHMANN H P, PIBIRI G E, et al. PHOBIC: Perfect Hashing with Optimized Bucket Sizes and Interleaved Coding[C/OL]//LIPIcs: Proceedings of the 32nd Annual European Symposium on Algorithms (ESA 2024): vol. 308. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024: 69:1-69:17. DOI: 10.4230/LIPIcs.ESA.2024.69.
 - [13] LEHMANN H P, SANDERS P, WALZER S, et al. Combined Search and Encoding for Seeds, with an Application to Minimal Perfect Hashing[C/OL]//LIPIcs: Proceedings of the 33rd Annual European Symposium on Algorithms (ESA 2025): vol. 351. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025: 109:1-109:18. DOI: 10.4230/LIPIcs.ESA.2025.109.
 - [14] KURPICZ F, LEHMANN H P, SANDERS P. PaCHash: Packed and Compressed Hash Tables[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2023). SIAM, 2023: 162-175. DOI: 10.1137/1

- .9781611977561.ch14.
- [15] AAMAND A, KNUDSEN J B T, THORUP M. Load Balancing with Dynamic Set of Balls and Bins[C/OL]//Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC 2021). ACM, 2021: 1262-1275. DOI: 10.1145/3406325.3451107.
 - [16] BANSAL N, KUSZMAUL W. Balanced Allocations: The Heavily Loaded Case with Deletions[C/OL]//Proceedings of the 63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS 2022). IEEE, 2022: 801-812. DOI: 10.1109/FOCS54457.2022.00081.
 - [17] MITZENMACHER M, RICHA A, SITARAMAN R. The Power of Two Random Choices: A Survey of Techniques and Results[G]//Handbook of Randomized Computing. Springer, 2001: 255-312.
 - [18] PAGH R, PAGH A, RUZIC M. Linear Probing with Constant Independence [J/OL]. SIAM Journal on Computing, 2009, 39(3): 1107-1120. DOI: 10.1137/070702278.
 - [19] LUBY M, RACKOFF C. How to Construct Pseudorandom Permutations from Pseudorandom Functions[J/OL]. SIAM Journal on Computing, 1988, 17(2): 373-386. DOI: 10.1137/0217022.
 - [20] CARTER L, WEGMAN M N. Universal Classes of Hash Functions[J/OL]. Journal of Computer and System Sciences, 1979, 18(2): 143-154. DOI: 10.1016/0022-0000(79)90044-8.
 - [21] THORUP M, ZHANG Y. Tabulation-Based 5-Independent Hashing with Applications to Linear Probing and Second Moment Estimation[J/OL]. SIAM Journal on Computing, 2012, 41(2): 293-331. DOI: 10.1137/100800774.
 - [22] BAYER R, MCCREIGHT E M. Organization and Maintenance of Large Ordered Indexes[J/OL]. Acta Informatica, 1972, 1(1): 173-189. DOI: 10.1007/BF00288683.
 - [23] PAGH R. Hashing with Limited Independence[C]//Proceedings of the 12th An-

- nual ACM-SIAM Symposium on Discrete Algorithms (SODA 2001). SIAM, 2001: 830-839.
- [24] JACOBSON G. Space-efficient Static Trees and Graphs[C/OL]//Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS 1989). IEEE, 1989: 549-554. DOI: 10.1109/SFCS.1989.63533.
- [25] CONWAY A, FARACH-COLTON M, KUSZMAUL W. A Unified Approach to Dynamic Optimality for Linear Probing and Robin Hood Hashing[C/OL]//LIPIcs: Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018): vol. 112. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018: 29:1-29:14. DOI: 10.4230/LIPIcs.ESA.2018.29.
- [26] MORRISON D R. PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric[J/OL]. Journal of the ACM, 1968, 15(4): 514-534. DOI: 10.1145/321066.321078.

致 谢

行文至此，落笔为终。感谢遇到的导师、感谢身边的每一个同学、感谢遇到的每一个人。凡是过往，皆为序章，很喜欢看到的一句话，人生如棋，落子无悔。愿我们跃入人海，各自灿烂。